

MaduraApp

*Sistema de Análisis de Madurez Agrícola mediante Visión
Computacional*

Documentación de Proyecto — Estado de Avance 3

Asignatura	Taller Aplicado de Programación (TPY1101)
Sección	001D
Estudiante	Claudio Vicente Aro Kath — RUT 22.022.498-8
Docente guía	José Ignacio Campos Arévalo
Fecha	Junio 2026

Repositorio

github.com/apotheosisss/MaduraApp-Produccion

Índice

Índice	3
Resumen ejecutivo	4
Parte 1 — Descripción y Gestión del Proyecto.....	5
1. Contexto del proyecto e impacto en el proceso de negocio.....	5
2. Descripción del problema u oportunidad.....	5
3. Diagnóstico del problema.....	6
4. Objetivos del proyecto	6
5. Situación inicial y alcance del proyecto.....	7
6. Planificación del proyecto (Carta Gantt)	7
7. Arquitectura tecnológica y justificación de servicios cloud	8
8. Conceptualización y atributos de calidad.....	8
9. Estrategia de certificación y revisiones parciales	9
Parte 2 — Diseño de Arquitectura, Ambientes y Plan de Pruebas.....	10
10. Diseño de la solución: Modelo 4+1 de Kruchten.....	10
11. Configuración de servidores (paridad Dev/Prod).....	12
12. Procedimiento de copia de seguridad y restauración (RDS PostgreSQL).....	13
Parte 3 — Aseguramiento de Calidad (Evaluación 3).....	14
13. Plan de pruebas	14
14. Base de datos de pruebas	14
15. Aplicación de pruebas y resultados.....	14
16. Pruebas de rendimiento y concurrencia	15
17. Mejoras al producto	15
18. Evidencia de control de versiones.....	15
Parte 4 — Conclusión, Aprendizajes y Anexos	16
19. Conclusión	16
20. Aprendizajes	16
21. Anexos	16

Resumen ejecutivo

MaduraApp es un sistema de visión computacional que clasifica el estado de madurez de cuatro frutas climatéricas en tres categorías y entrega recomendaciones en tiempo real, atacando una causa raíz del 20–40% de pérdidas post-cosecha en Chile (FAO/ODEPA). El producto integra una app Android nativa, una API REST en FastAPI y un modelo YOLO26n ($mAP@50 = 0,92$). Este informe documenta la gestión del proyecto, el diseño arquitectónico (modelo 4+1), la configuración de ambientes y el aseguramiento de calidad: 57 pruebas automatizadas (76% de cobertura), pruebas de rendimiento bajo concurrencia, endurecimiento de seguridad OWASP y 15 mejoras trazables. La solución fue desplegada y verificada en AWS (EC2 + RDS PostgreSQL).

Parte 1 — Descripción y Gestión del Proyecto

1. Contexto del proyecto e impacto en el proceso de negocio

Cliente objetivo: consumidores finales, ferias libres y pequeños productores de frutas climatéricas en Chile.

Proceso de negocio afectado — «Decisión de consumo/procesamiento de la fruta»: actualmente la decisión de cuándo consumir, vender o procesar una fruta se toma de forma subjetiva (vista y tacto), sin criterio objetivo. Esto provoca que la fruta se deseche por considerarla «pasada» cuando aún es apta, o que se consuma fuera de su punto óptimo.

Impacto de la solución: MaduraApp transforma ese proceso entregando, en ~1 segundo desde un smartphone, un diagnóstico objetivo del estado de madurez (precisión 92%) con una recomendación de acción. Al objetivar la decisión, ataca directamente la causa de una fracción del 20–40% de pérdidas post-cosecha que reporta FAO/ODEPA, sin requerir equipamiento ni conocimiento agronómico del usuario.

2. Descripción del problema u oportunidad

Las pérdidas post-cosecha en frutas climatéricas (aguacate, plátano, tomate, mango) alcanzan en Chile entre 20% y 40% de la producción (FAO/ODEPA). Una causa raíz es la ausencia de un criterio objetivo y accesible para determinar el punto óptimo de consumo o procesamiento.

Análisis de causas y efectos (Ishikawa)

- Método / Personas: evaluación subjetiva por vista y tacto; falta de conocimiento agronómico.
- Tecnología: inexistencia de herramientas accesibles que clasifiquen la madurez objetivamente.
- Información: ausencia de recomendaciones contextualizadas por fruta y estado.
- Efecto central: decisiones erróneas de consumo/descarte → desperdicio de fruta apta.

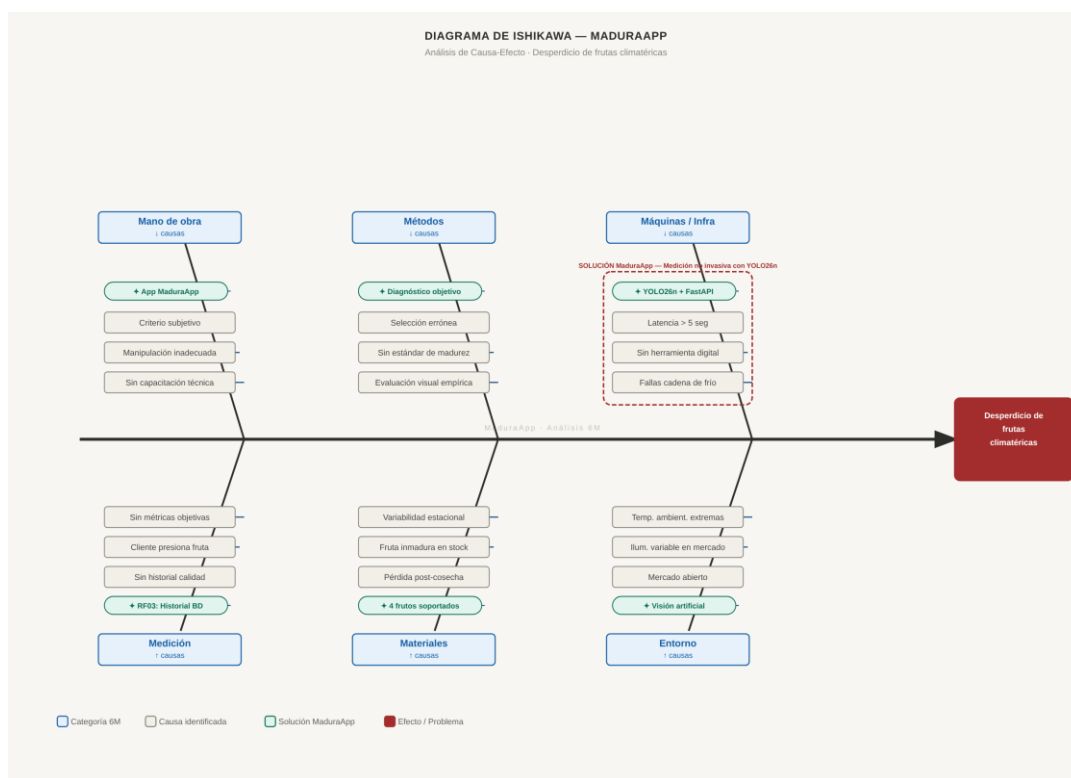


Figura 1. Diagrama de Ishikawa — análisis de causas.

Oportunidad de mejora: diseñar una aplicación móvil con visión computacional que clasifique la madurez en tiempo real y entregue recomendaciones, democratizando un criterio objetivo antes reservado a expertos.

3. Diagnóstico del problema

La solución ataca las tres ramas del problema: (1) entrega un criterio objetivo y medible (el modelo de IA), (2) lo hace accesible desde cualquier smartphone sin hardware especial, y (3) agrega recomendaciones que cierran la brecha de conocimiento del usuario. El resultado esperado es una reducción del descarte de fruta apta y un consumo en el punto óptimo.

4. Objetivos del proyecto

Objetivo general

Desarrollar y validar un sistema de visión computacional (app Android + API + modelo de IA) que clasifique el estado de madurez de cuatro frutas climatéricas en tres categorías, entregando un diagnóstico y una recomendación en tiempo real, con calidad y seguridad de nivel productivo.

Objetivos específicos (cuantificables — SMART)

ID	Objetivo	Meta	Obtenido	Estado
OE-01	Precisión del modelo (mAP@50)	≥ 0,75	0,9229	✓
OE-02	Rendimiento (inferencia / E2E)	< 400 ms / < 1 s	~200 ms / ~600-700 ms	✓
OE-03	Pruebas y cobertura	≥ 57 casos · ≥ 75%	57/57 · 76%	✓

OE-04	Estabilidad bajo carga	0 errores @ 50 concurrentes	0 errores	✓
OE-05	Eficiencia del modelo	< 10 MB	5,2 MB	✓
OE-06	Seguridad de datos personales	OWASP · cifrado	Cumplido	✓

5. Situación inicial y alcance del proyecto

Alcance técnico

Diseño, desarrollo, pruebas y despliegue de: (a) app Android nativa (Kotlin + CameraX) con captura/galería, selector de fruta, semáforo de madurez, historial offline, autenticación y feedback; (b) API REST FastAPI con inferencia YOLO26n, JWT, historial y feedback; (c) modelo YOLO26n entrenado para 4 frutas × 3 estados.

Entregables

- Código fuente (app + backend) versionado en Git.
- Modelo entrenado (yolo26n_maduraapp.pt, mAP@50 = 0,92).
- Suite de 57 pruebas automatizadas + reporte de cobertura.
- Documentación: 4+1, plan de pruebas, configuración de ambientes, backup, informe técnico.
- Pipeline CI (GitHub Actions) y despliegue en AWS.

Supuestos

- Disponibilidad de los créditos del AWS Academy Learner Lab para el despliegue de pruebas.
- El usuario final dispone de un smartphone Android (API 29+) con cámara e internet.
- Las imágenes corresponden a una de las cuatro frutas en iluminación razonable.

Restricciones

- Temporal: desarrollo acotado a las 3 evaluaciones parciales de la asignatura.
- Infraestructura: el Learner Lab es efímero, sin GPU y con rol IAM restringido (LabRole), lo que condiciona el modelo de servicio cloud.
- Alcance del modelo: limitado a 4 frutas climatéricas y 3 estados.

6. Planificación del proyecto (Carta Gantt)

El proyecto se ejecutó en tres iteraciones alineadas a las evaluaciones parciales (EP1: definición; EP2: producto; EP3: aseguramiento de calidad y despliegue).

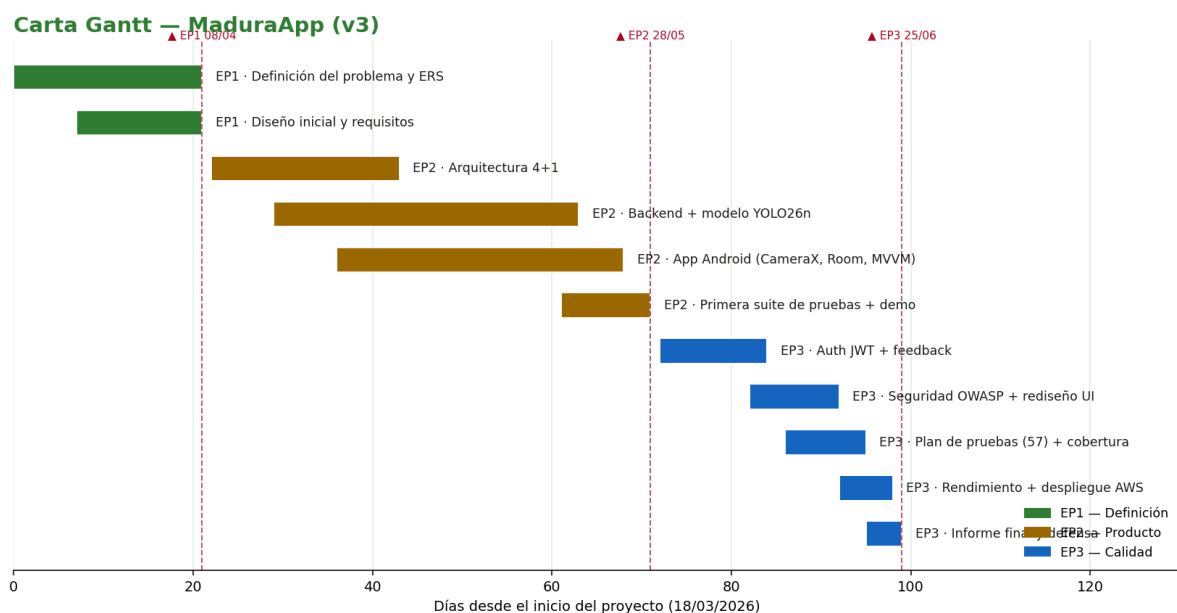


Figura 2. Carta Gantt v3 — tres iteraciones del proyecto.

7. Arquitectura tecnológica y justificación de servicios cloud

Stack: Android nativo (Kotlin + CameraX + Room + MVVM) · FastAPI (Python 3.12 async) · YOLO26n · SQLAlchemy + Alembic · PostgreSQL (producción) / SQLite (desarrollo) · Docker · GitHub Actions.

Modelo	Servicio	Justificación
IaaS	AWS EC2 (t3.small, Docker)	Cómputo del backend; elegido por restricción del Learner Lab, que no habilita PaaS con roles personalizados. EC2 da control total del contenedor.
DBaaS	AWS RDS PostgreSQL (prod) · SQLite (dev)	Base de datos gestionada: respaldos y parches por AWS; habilita pg_dump/pg_restore.
SaaS	GitHub Actions (CI)	Ejecuta la suite de pruebas en cada push sin administrar infraestructura.
Contenedores	Docker	Paridad de ambientes dev/prod: la misma imagen corre en local y en EC2.

Nota de honestidad técnica: el formato de referencia valora PaaS (p. ej. App Runner). Se usó EC2 (IaaS) porque el AWS Academy Learner Lab no permite desplegar en PaaS con los roles disponibles. La app es config-driven (la BD se elige por DB_URL), por lo que el salto SQLite → RDS PostgreSQL es un cambio de configuración, no de código.

8. Conceptualización y atributos de calidad

- Integridad: validación de imagen, firma JWT, esquema versionado (Alembic).
- Confiabilidad y seguridad: OWASP (JWT, bcrypt, HTTPS, AES-256); 0 errores bajo 50 concurrentes.
- Precisión y oportunidad: mAP@50 = 0,92; inferencia ~200 ms; recomendaciones por estado.
- Mantenibilidad: arquitectura 4+1, 57 pruebas, CI, Conventional Commits.

- Usabilidad: Material Design 3 con modo oscuro y semáforo de madurez.

9. Estrategia de certificación y revisiones parciales

El producto se considera certificado al cumplir simultáneamente los criterios de aceptación:

Criterio	Meta	Resultado
Precisión del modelo	mAP@50 $\geq$ 0,75	0,9229 
Suite de pruebas	100% en verde	57/57 
Cobertura de código	$\geq$ 75% núcleo	76% 
Estabilidad bajo carga	0 errores @ 50	0 
Build de la app	APK compila	
Seguridad	0 secretos · OWASP	

Revisiones parciales: EP1 (problema, requisitos, diseño), EP2 (arquitectura 4+1 y producto), EP3 (aseguramiento de calidad: pruebas, cobertura, rendimiento, seguridad, mejoras y despliegue).

Parte 2 — Diseño de Arquitectura, Ambientes y Plan de Pruebas

10. Diseño de la solución: Modelo 4+1 de Kruchten

10.1 Vista de Escenarios (Casos de Uso — el «+1»)

Define el producto desde la lógica de negocio. Escenario principal: el usuario inicia sesión, selecciona la fruta, captura o sube una foto; la app la envía al backend, que ejecuta la inferencia YOLO26n, persiste el escaneo y devuelve el diagnóstico (estado, color, recomendación); el usuario puede calificar el resultado y consultar su historial, incluso sin conexión.

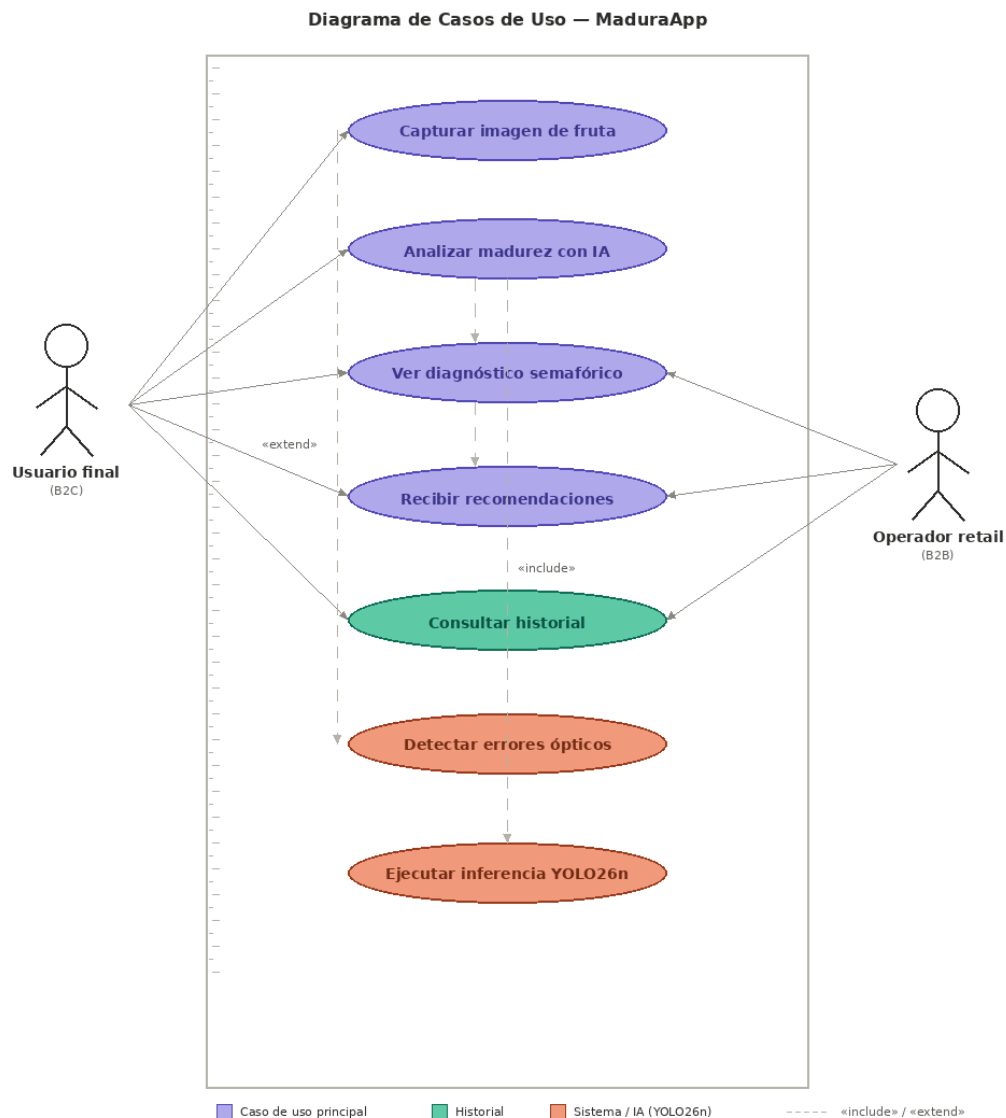


Figura 3. Diagrama de casos de uso.

10.2 Vista Lógica (diseño del software)

Representa la estructura interna: la app sigue el patrón MVVM (Activity/ViewModel/Repository) con una capa de datos que integra la API (Retrofit) y el cache local (Room); el backend separa routers, services, models y schemas, con la lógica de clasificación de madurez en el servicio de inferencia.

Diagrama de Clases UML — MaduraApp (v2)

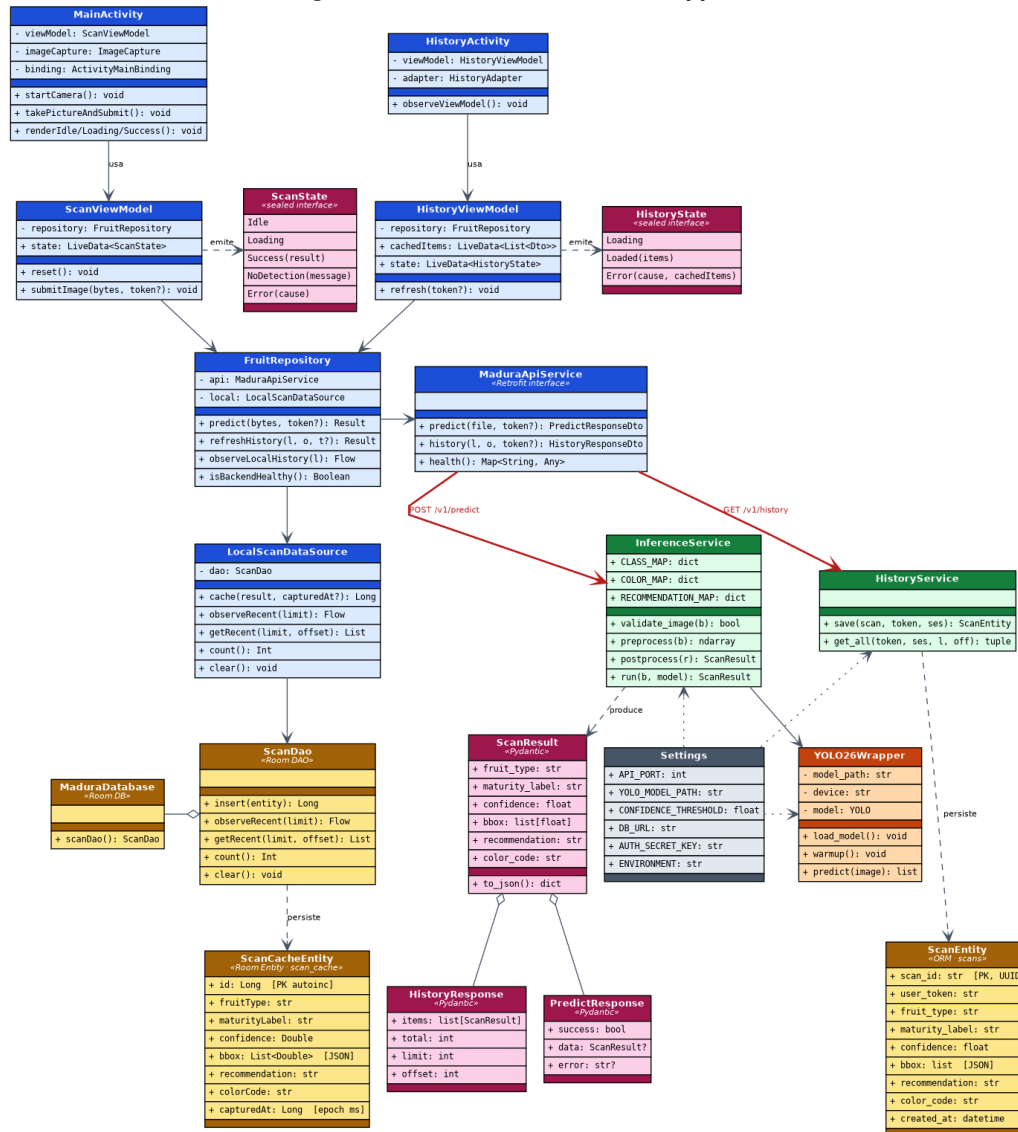


Figura 4. Diagrama de clases.

10.3 Vista de Procesos (flujo y concurrencia)

El backend es asíncrono (async/await): la inferencia se ejecuta en un hilo aparte (asincio.to_thread) para no bloquear el bucle de eventos, permitiendo atender solicitudes concurrentes. La app usa corrutinas de Kotlin para las operaciones de red y base de datos.

10.4 Vista de Desarrollo (estructura del código)

El repositorio se organiza en backend/ (FastAPI: app/routers, services, models, schemas, core; tests; alembic) y frontend/ (app Android Kotlin con ui, data, di). El control de versiones usa Git con Conventional Commits e integración continua en GitHub Actions.

10.5 Vista de Despliegue (infraestructura)

La app Android se comunica por HTTPS (Bearer JWT) con el backend FastAPI, que corre en un contenedor Docker sobre AWS EC2 (t3.small) y persiste en AWS RDS PostgreSQL (misma VPC). El modelo de datos se muestra en la Figura 5.

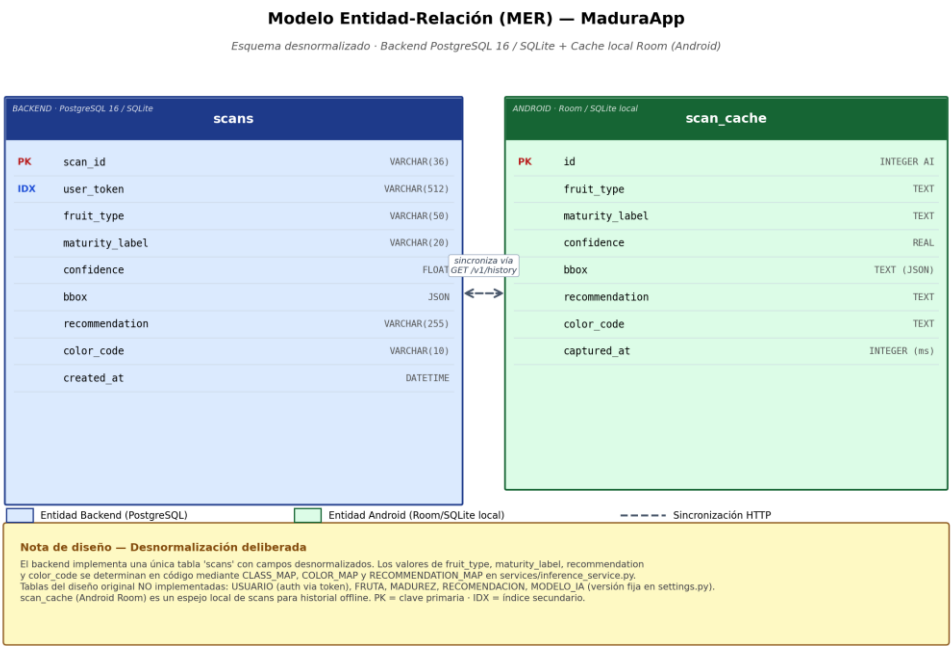


Figura 5. Modelo entidad-relación (datos).

11. Configuración de servidores (paridad Dev/Prod)

El contenedor Docker es idéntico en desarrollo y producción; solo cambian las variables de entorno.

11.1 Receta del servidor — Dockerfile

```
FROM python:3.12-slim · instala libgl1/libglib2.0-0 (visión) · pip install -r requirements.txt ·  
EXPOSE 8000 · CMD uvicorn app.main:app --host 0.0.0.0 --port 8000.
```

11.2 Variables de entorno

Variable	Desarrollo	Producción
ENVIRONMENT	development	production
JWT_SECRET_KEY	dev	secreto fuerte (≥32, por entorno)
DB_URL	sqlite+aiosqlite:///./maduraapp_dev.db	postgresql+asyncpg://...rds.amazonaws.com:5432/maduraapp
CORS_ORIGINS	*	dominios acotados

Los secretos nunca se versionan (solo .env.example). En producción la app rechaza arrancar con el secreto por defecto (validación en core/config.py).

11.3 Despliegue verificado (EC2 + RDS)

docker build → docker run --env-file .env.prod → docker exec alembic upgrade head (crea las tablas en RDS) → curl /v1/health. El despliegue fue verificado el 25/06/2026: las migraciones se aplicaron sobre RDS PostgreSQL (Context impl PostgresqlImpl) y el flujo registro → login se probó extremo a extremo contra la base PostgreSQL.

12. Procedimiento de copia de seguridad y restauración (RDS PostgreSQL)

Con la base de datos en RDS PostgreSQL, los respaldos usan las herramientas nativas de Postgres:

- Respaldo: `pg_dump -h <rds-endpoint> -U <user> -d maduraapp -F c -b -v -f backup.backup`
- Preparar staging: `psql ... -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"`
- Restauración: `pg_restore -h <staging> -U <user> -d maduraapp_test -v backup.backup`

Adicionalmente, AWS RDS realiza snapshots automáticos diarios y permite point-in-time recovery, por lo que el respaldo manual complementa la protección gestionada del servicio.

Parte 3 — Aseguramiento de Calidad (Evaluación 3)

13. Plan de pruebas

Se confeccionó un plan de 57 casos automatizados (38 backend + 19 Android), clasificados en cuatro tipos: validación (cumple requisitos), verificación (calidad), seguridad (OWASP) y operacional (funciona en su entorno). Resumen por categoría:

Categoría	Casos	Estado
Verificación del estado de madurez (backend)	21	✓
Validación funcional y seguridad (backend)	17	✓
Validación funcional (Android)	19	✓
Subtotal automatizado	57	✓ 57/57
Verificación de calidad / KPI	6	✓
Operacional	5	✓ parcial

13.1 Verificación del estado de madurez de la fruta (prioridad)

21 casos verifican el núcleo del producto: que cada detección del modelo se traduzca en el estado de madurez, color de semáforo y recomendación correctos, además de la lógica de umbral y de filtro de fruta. Se simula la salida de YOLO para probar la lógica sin depender de los pesos.

ID	Funcionalidad	Esperado	Estado
CP-EM-01	Consistencia mapas (12 clases)	color+recom. por clase	✓
CP-EM-02	Semáforo por estado	green/yellow/red	✓
CP-EM-03..14	Estado correcto por clase (12)	fruta+estado+color	✓
CP-EM-15..20	Confianza, umbral, filtro, recomendación	comportamiento correcto	✓

14. Base de datos de pruebas

Las pruebas usan una base efímera y aislada: SQLite in-memory para el backend, creada y destruida por test-module, con fixtures que generan un usuario, un JWT real e imágenes JPEG sintéticas (el modelo YOLO se reemplaza por un mock). En Android se usan dobles de prueba (MockK) de la API y de Room. Ningún dato real ni secreto se usa en pruebas; las contraseñas de prueba se hashean con bcrypt igual que en producción.

15. Aplicación de pruebas y resultados

Ejecutadas el 25/06/2026: backend 38/38 (pytest) y Android 19/19 (gradlew testDebugUnitTest). Total: 57/57 en verde.

Suite	Framework	Pruebas	Resultado
Backend (FastAPI)	pytest	38	✓ 38/38
Android — Repositorio/cache	MockK	9	✓ 9/9
Android — ViewModels	JUnit	10	✓ 10/10
TOTAL		57	✓ 57/57

15.1 Cobertura de código

Medida con pytest --cov=app sobre los 38 tests del backend: 76% total (410 sentencias, 99 sin cubrir). Los módulos de lógica crítica están altamente cubiertos (seguridad 100%, modelos 100%, routers 90-94%, inferencia alta); la brecha se concentra en código de infraestructura difícil de probar por unidad (wrapper del modelo real 29%, lifespan de arranque).

16. Pruebas de rendimiento y concurrencia

Ambiente: cliente (PC del estudiante, Chile) → servidor AWS EC2 t3.small (us-east-1), por internet. Herramienta: script httpx + ThreadPoolExecutor.

Métrica	Resultado
Inferencia /predict (procesamiento)	~200 ms
Latencia extremo a extremo (Chile → AWS)	~600-700 ms (incluye RTT de red)
Estabilidad bajo concurrencia	0 errores hasta 50 usuarios simultáneos
Cuello de botella	CPU durante la inferencia (sin GPU)

17. Mejoras al producto

Los hallazgos de las pruebas y de una auditoría OWASP originaron 15 mejoras trazables a commits, mapeadas a los cinco estándares de calidad de la industria.

Estándar	Mejoras destacadas
Seguridad	Secreto JWT por entorno, CORS acotado, política de contraseña, HTTPS forzado, token cifrado AES-256, hashing bcrypt
Usabilidad	Rediseño Material 3: íconos vectoriales, modo oscuro, feedback táctil, tipografía consistente
Corrección	Suite y build a verde tras integración; integración continua (CI)
Compleitud	Autenticación + feedback integrados y probados end-to-end
Pertinencia	Diagnóstico y recomendaciones ajustados al dominio agrícola

18. Evidencia de control de versiones

Git + GitHub, Conventional Commits, ramas de feature, CI (backend_ci.yml) que ejecuta la suite en cada push, y copias de configuración versionadas sin secretos (.env.example, requirements.txt, migraciones Alembic, build.gradle.kts, .gitignore).

Parte 4 — Conclusión, Aprendizajes y Anexos

19. Conclusión

MaduraApp alcanzó el estado de producto funcional, probado, seguro y desplegado. Se cumplieron los seis objetivos SMART: el modelo supera el KPI de precisión (0,92), el sistema responde en ~200 ms y soporta 50 usuarios concurrentes sin errores, la suite de 57 pruebas pasa al 100% con 76% de cobertura, y los datos personales se protegen con cifrado en tránsito y reposo. La arquitectura se desplegó y verificó en AWS (EC2 + RDS PostgreSQL), con la documentación coherente con la implementación real.

20. Aprendizajes

- La infraestructura real rara vez coincide con el plan, y eso debe documentarse: un diagrama que contradice el Dockerfile o las pruebas es un defecto de calidad.
- Las restricciones del entorno (Learner Lab efímero, sin GPU, IAM restringido) definen la arquitectura tanto como los requisitos; conviene justificar las decisiones con honestidad.
- Una métrica sin número no es un objetivo: reformular metas vagas como objetivos SMART permitió evaluar el producto con evidencia medible; la cobertura del 76% reveló dónde faltaban pruebas.
- La prueba de rendimiento aísla cuellos de botella reales: el límite es la CPU de inferencia, no la red ni la base de datos.
- La separación dev/prod por configuración (DB_URL) permitió usar SQLite en pruebas y PostgreSQL en producción sin cambiar código.
- Las pruebas son una red de seguridad real: detectaron regresiones al integrar la autenticación.
- La honestidad técnica frente al usuario (no etiquetar como «extremo a extremo») es parte de la ética y de la calidad del producto.

21. Anexos

A. Contrato de la API REST

Método	Ruta	Respuesta
POST	/v1/auth/register	201 {access_token, user_id, username}
POST	/v1/auth/login	200 {access_token, ...}
GET	/v1/auth/me	200 {user_id, username, email}
POST	/v1/predict	200 {success, data:{fruit_type, maturity_label, confidence, bbox, recommendation, color_code, scan_id}}
GET	/v1/history	200 {items[], total, limit, offset}
POST	/v1/feedback	201
GET	/v1/health	200 {status, model, version, model_loaded}

B. Esquema de base de datos

users(user_id PK, username UNIQUE, email UNIQUE, hashed_password, is_active, created_at) ·
scans(id PK, user_id FK, fruit_type, maturity_label, confidence, bbox, recommendation,

color_code, created_at) · scan_feedback(id PK, scan_id FK, user_id, rating 1-5, created_at).
Gestionado con Alembic.

C. Glosario

YOLO26n · mAP@50 · FastAPI · JWT · bcrypt · AES-256 · OWASP (A01/A02/A07) · MVVM · Room · CameraX · Alembic · CI · IaaS/DBaaS/SaaS · cobertura de código · fixture · mock.

D. Referencias

- FAO / ODEPA — Estadísticas de pérdidas post-cosecha en frutas (Chile).
- OWASP Foundation — OWASP Top 10 (2021).
- Ultralytics — YOLO documentation.
- Google — Material Design 3 guidelines.
- AWS — RDS PostgreSQL y EC2 documentation.